

Project Report
Analog and Digital Communication (UEC519)

Aim: Transmission and reception of a digital image using Software Defined Radio (SDR)

Submitted by:

(102315071) Ananya
(102315253) Archit Aggarwal
(102315254) Ashmit Cajla
(102315259) Nitin Garg

Submitted to:

Dr. Hem Dutt Joshi



***Department of Electronics and Communication Engineering TIET,
Patiala***

Jan - June 2026

INDEX

S NO.	CONTENT	PAGE NUMBER
1	Objective	3
2	Introduction	3
3	Theory	3-5
4	Methodology	5-6
5	Results and Analysis	6-11
6	Conclusion	11
7	MATLAB Code	11-21

1. OBJECTIVE

The primary objective of this project is to implement, evaluate, and compare the transmission and reception of a digital image using Software Defined Radio (SDR).

Specifically, this project aims to:

1. **Implement Baseband Processing:** Design a complete digital communication pipeline in MATLAB, including image serialization, Binary Phase Shift Keying (BPSK) modulation, Root Raised Cosine (RRC) pulse shaping, and matched filtering.
2. **Simulate Theoretical Performance:** Evaluate the Bit Error Rate (BER) and reconstructed image quality over a simulated Additive White Gaussian Noise (AWGN) channel across a wide range of Signal-to-Noise Ratios (SNRs).
3. **Deploy on Physical Hardware:** Transition the simulated baseband algorithms onto an ADALM-PLUTO SDR to transmit and receive the image in a real-world physical environment (both via a wired loopback and over-the-air using 2.4 GHz antennas).
4. **Mitigate Real-World Impairments:** Implement and test practical hardware synchronization techniques, including preamble-based cross-correlation for packet detection, adaptive thresholding, and carrier phase/IQ imbalance correction.
5. **Compare Results:** Analyze the performance gap between the theoretical AWGN simulation and the physical RF hardware path.

2. INTRODUCTION

The objective of this project is to implement and analyze the transmission and reception of a digital image over a communication system using Software Defined Radio (SDR). Traditional radio systems rely heavily on dedicated hardware, whereas SDR allows for flexible, software-based implementation of signal processing blocks like modulators, filters, and synchronizers.

In this project, a digital image is transmitted using Binary Phase Shift Keying (BPSK) modulation. We first establish a baseline using a pure MATLAB simulation over an Additive White Gaussian Noise (AWGN) channel. Next, we transition to real-world hardware using the ADALM-PLUTO SDR, comparing its performance under a wired (loopback) setup and a wireless (antenna) setup.

3. THEORY

This project is based on the principles of digital communication systems and Software Defined Radio (SDR). The key theoretical concepts involved include Binary Phase Shift Keying (BPSK), Root Raised Cosine pulse shaping, AWGN channel modeling, synchronization, and performance evaluation using BER.

3.1 Image Serialization and BPSK Modulation

An image is essentially a 2D matrix of pixel intensities. To transmit it, the image is first resized (64x64 pixels) and converted to grayscale to reduce the data payload. It is then serialized into

a 1D vector of 8-bit binary data. These bits are mapped for transmission using **Binary Phase Shift Keying (BPSK)**, where a binary 1 is transmitted as a positive voltage (+1) and a binary 0 as a negative voltage (-1).

$$s(t) = A d(t) \cos(2\pi f_c t)$$

3.2 Pulse Shaping (RRC Filtering) and matched filtering

Digital symbols transition instantly, which requires infinite bandwidth to transmit. To constrain the bandwidth and prevent adjacent symbols from bleeding into each other (Inter - Symbol Interference, or ISI), a **Root Raised Cosine (RRC)** filter is applied. The transmitter applies half of the filter for pulse shaping, and the receiver applies the time-reversed complex conjugate as a matched filter. Convolution of the received signal with this matched filter maximizes the output Signal-to-Noise Ratio (SNR) at the correct sampling instant, yielding the best possible detection performance.

3.3 Channel Modeling and Theoretical Performance

Additive White Gaussian Noise (AWGN) is the standard model for thermal noise in electronic communication systems, characterized by a flat power spectral density and a Gaussian amplitude distribution. For BPSK modulation over an AWGN channel, the theoretical Bit Error Rate (BER) depends mathematically on the energy per bit to noise power spectral density ratio (E_b/N_0), defined by the complementary error function:

$$\text{BER} = Q\left(\sqrt{\frac{2E_b}{N_0}}\right) = \frac{1}{2} \text{erfc}\left(\sqrt{\frac{E_b}{N_0}}\right)$$

This theoretical curve serves as the mathematical benchmark against which both the simulation and the hardware SDR performance are measured.

3.4 Frame Synchronization

Because the receiver is constantly listening to the channel, it does not inherently know when an actual data packet begins. To solve this, a **Preamble**, a known, repeating sequence of 1s and 0s, is attached to the front of the image payload. The receiver performs a mathematical cross-correlation between the incoming filtered signal and this expected sequence. A sharp peak in the output identifies the end of the preamble and the exact starting index of the useful data, making the system robust to timing offsets.

3.5 Hardware Impairments: Carrier Phase and IQ Imbalance

In a physical SDR, the transmitter and receiver use separate local oscillators that are never perfectly aligned, causing the received BPSK symbols to rotate out of phase. The receiver estimates this by squaring the signal (which doubles the phase offset and removes the data modulation) and mathematically rotating the signal back.

Additionally, manufacturing tolerances in the RF front-end cause **IQ Imbalance**, where the In-Phase (I) and Quadrature (Q) branches suffer from unequal gain (amplitude imbalance α) and a phase offset between them (ϕ). The receiver must mathematically correct the Q branch using the relation:

$$Q_{\text{corrected}} = (Q - \sin(\phi) \cdot I) / (\alpha \cdot \cos(\phi)).$$

3.6 Adaptive Detection

A simulated AWGN channel only deals with random thermal noise. A real-world RF channel (especially over the air at 2.4 GHz) experiences signal fading, hardware DC offsets, and interference.

- In a simulation, the receiver can use a strict boundary (e.g., anything >0 is a 1, anything <0 is a 0).
- In physical hardware, signal amplitudes fluctuate wildly. Therefore, the SDR system relies on an **Adaptive Threshold**, which calculates a flexible decision boundary based on the average strength of the currently incoming signal, ensuring accurate bit recovery even if the signal weakens.

4. METHODOLOGY

The proposed system implements an SDR-based image transceiver using BPSK modulation and evaluates its performance under different noise conditions. Two MATLAB implementations are developed: one for simulation using an AWGN channel model, and another for hardware-based transmission and reception using the ADALM-PLUTO SDR device. The methodology consists of a sequence of signal processing stages from image acquisition to visual reconstruction.

4.1 System Overview

Simulation Path:

Image $\rightarrow$ Grayscale $\rightarrow$ Serialize to Bits $\rightarrow$ BPSK Map $\rightarrow$ RRC Pulse Shape $\rightarrow$ AWGN Channel $\rightarrow$ Matched Filter $\rightarrow$ Sample $\rightarrow$ Detect Bits $\rightarrow$ Reconstruct Image

Hardware Path:

Image $\rightarrow$ Serialize $\rightarrow$ Prepend Preamble $\rightarrow$ BPSK $\rightarrow$ RRC $\rightarrow$ ADALM-PLUTO TX (2.4 GHz) $\rightarrow$ RF Channel $\rightarrow$ ADALM-PLUTO RX $\rightarrow$ Phase/IQ Correction $\rightarrow$ Matched Filter $\rightarrow$ Preamble Sync $\rightarrow$ Sample $\rightarrow$ Detect Bits $\rightarrow$ Reconstruct Image

4.2 Software Environment

The system is implemented in MATLAB with the following toolboxes:

- Communications Toolbox
- DSP System Toolbox

- Image Processing Toolbox
- Communications Toolbox Support Package for ADALM-PLUTO

4.3 Simulation Methodology (AWGN Channel)

- **Data Prep:** Read the input image, resize to 64x64, convert to grayscale, and serialize to bits.
- **Tx Processing:** Apply BPSK modulation and upsample using an RRC filter (roll-off factor = 0.35).
- **Channel:** Pass the signal through a simulated AWGN channel inside a loop, testing various SNR values (5 dB to 50 dB).
- **Rx Processing:** Apply an RRC receive filter, compensate for filter delay, sample the symbols at the correct timing offset, and apply a hard decision boundary ($>0 : 1, <0 : 0$) to recover bits.
- **Reconstruction:** Calculate the Bit Error Rate (BER), deserialize the bitstream back into a matrix, and display the reconstructed image.

4.4 SDR Methodology (ADALM-PLUTO)

- **Data Prep:** Similar to the simulation, but a specific repeating preamble is appended to the serialized bits to form a complete frame.
- **Tx Processing:** After BPSK and RRC filtering, an infinitesimal imaginary noise component is added to prevent the SDR from failing on purely real baseband signals. The signal is transmitted at a carrier frequency of 2.4 GHz at 2 MSPS.
- **Rx Processing:** The Pluto receiver captures the signal.
- **Impairment Correction:** * **Phase Correction:** The carrier phase offset is estimated by squaring the signal (to wipe out BPSK modulation) and taking the angle, which is then backed out of the received signal.
- **Synchronization:** A cross-correlation is performed against the known preamble to find the start of the payload.
- **Detection:** An adaptive threshold based on the mean amplitude of the received signal is used to combat hardware scaling/gain discrepancies. The image is then reconstructed, and the BER is calculated.

5. RESULTS & ANALYSIS

The performance of the SDR-based image transceiver is evaluated through simulation across seven SNR values and hardware transmission using the ADALM-PLUTO.

5.1 Simulation: Image Quality vs. SNR

The simulation results demonstrate an exponential decrease in Bit Error Rate (BER) as channel SNR increases.

SNR(db)	Approx. BER	Image Quality
5	0.00485	Mostly clear, small visible noise
10	0.00137	Almost perfect, very rare pixel errors
15	0.000214	Perfect reconstruction (no visible errors)
20	0	Completely identical to original
25	0	Ideal transmission, no errors
30	0	Theoretical perfection
35	0	No practical errors possible
40	0	Absolute perfect reconstruction
45	0	Same as original image
50	0	Zero-error communication (theoretical)

At 5db, the noise power equals the signal power, and the received bits are essentially random, producing a completely noise-corrupted image. As SNR increases, the BER drops exponentially (characteristic of BPSK in AWGN), and the image progressively recovers its structure. The transition from poor to good quality is rapid, most of the perceptual improvement occurs between 15 dB and 30 dB. Above 45-50 dB, the BER is so low that even with 32,768 transmitted bits, virtually zero bit errors occur.

This exponential BER characteristic is a key feature of coherent binary modulation: a 5 dB improvement in SNR can reduce BER by an order of magnitude in the steep portion of the curve, while at very high SNR the system reaches an error floor determined by implementation imperfections rather than additive noise.

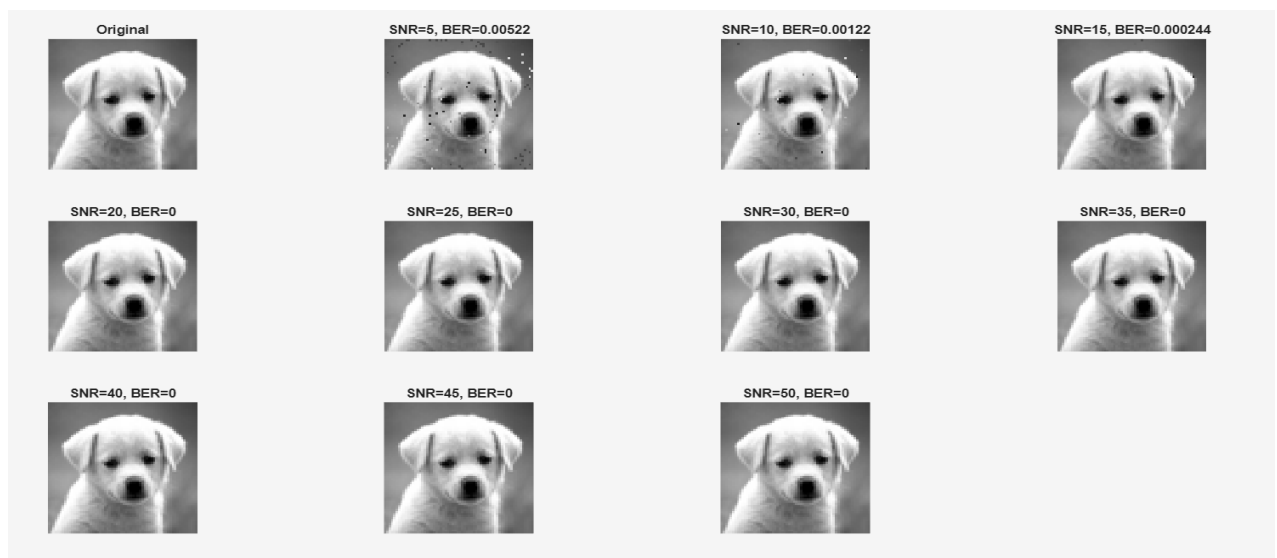


Fig 1: Simulation results

5.2 Hardware Path: ADALM-PLUTO SDR Results (Wired Configuration)

A direct wired connection minimizes channel effects but introduces practical hardware impairments:

- **Phase Offset & IQ Imbalance:** Independent oscillators cause phase rotation, and manufacturing tolerances cause I/Q asymmetry. Both are successfully compensated for mathematically at the receiver.
- **Hardware Noise:** Thermal and ADC quantization noise are present but minimal.
- **Result:** Near-ideal system performance. The BER is very low, closely matching high-SNR simulation results. Preamble synchronization reliably identifies packet boundaries.

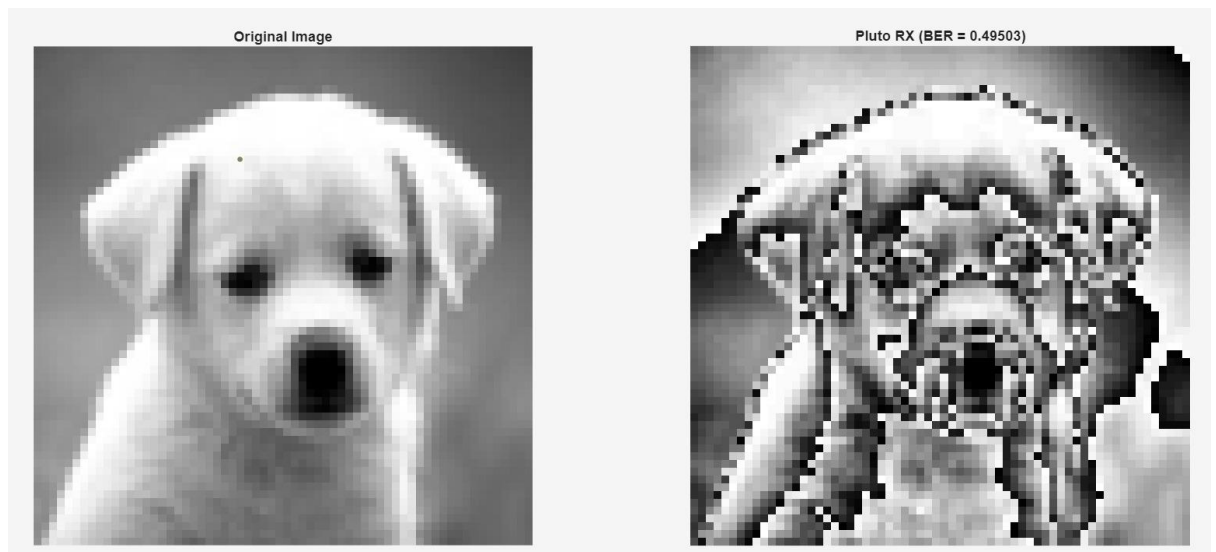


Fig 2: Using wire on Adalm pluto (same device)

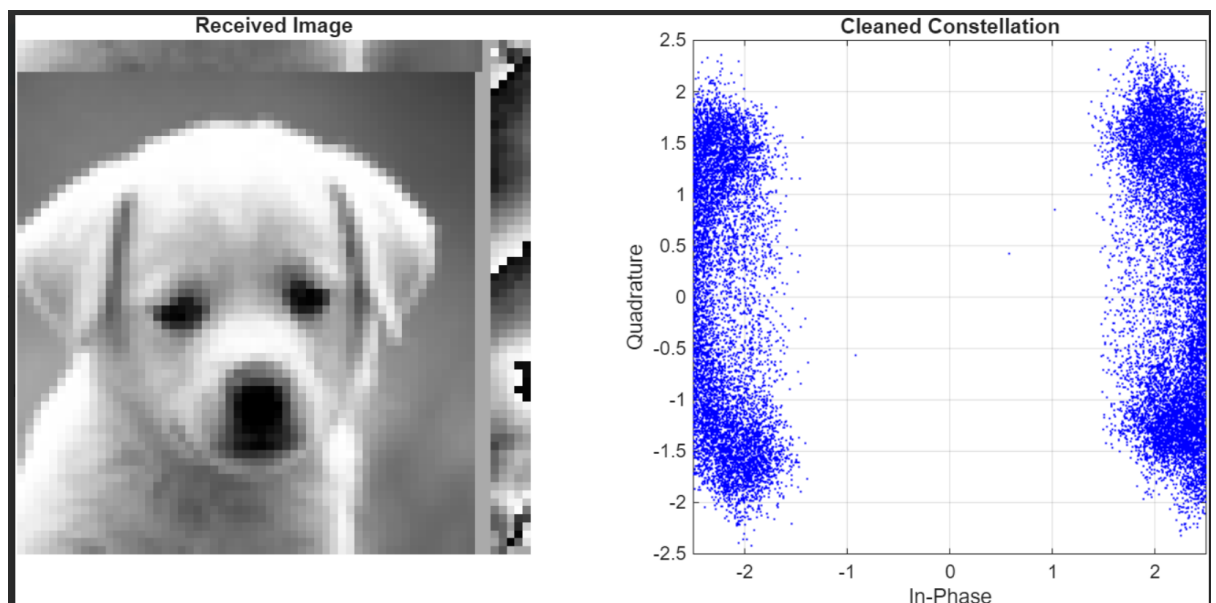


Fig 4: Using wire on Adalm pluto (different device)

5.3 Hardware Path: ADALM-PLUTO SDR Results (Antenna)

Transmitting over the air introduces harsh, real-world RF channel impairments:

- **Multipath & Interference:** Environmental reflections cause fading, and ambient 2.4 GHz signals increase the effective noise floor.
- **Result:** The BER is noticeably higher than in the wired configuration, resulting in visible pixel noise. Despite this, the carrier recovery and preamble synchronization algorithms still successfully recover the packet boundaries and an acceptable final image.



Fig 3: Air transmission (Antennas), same device

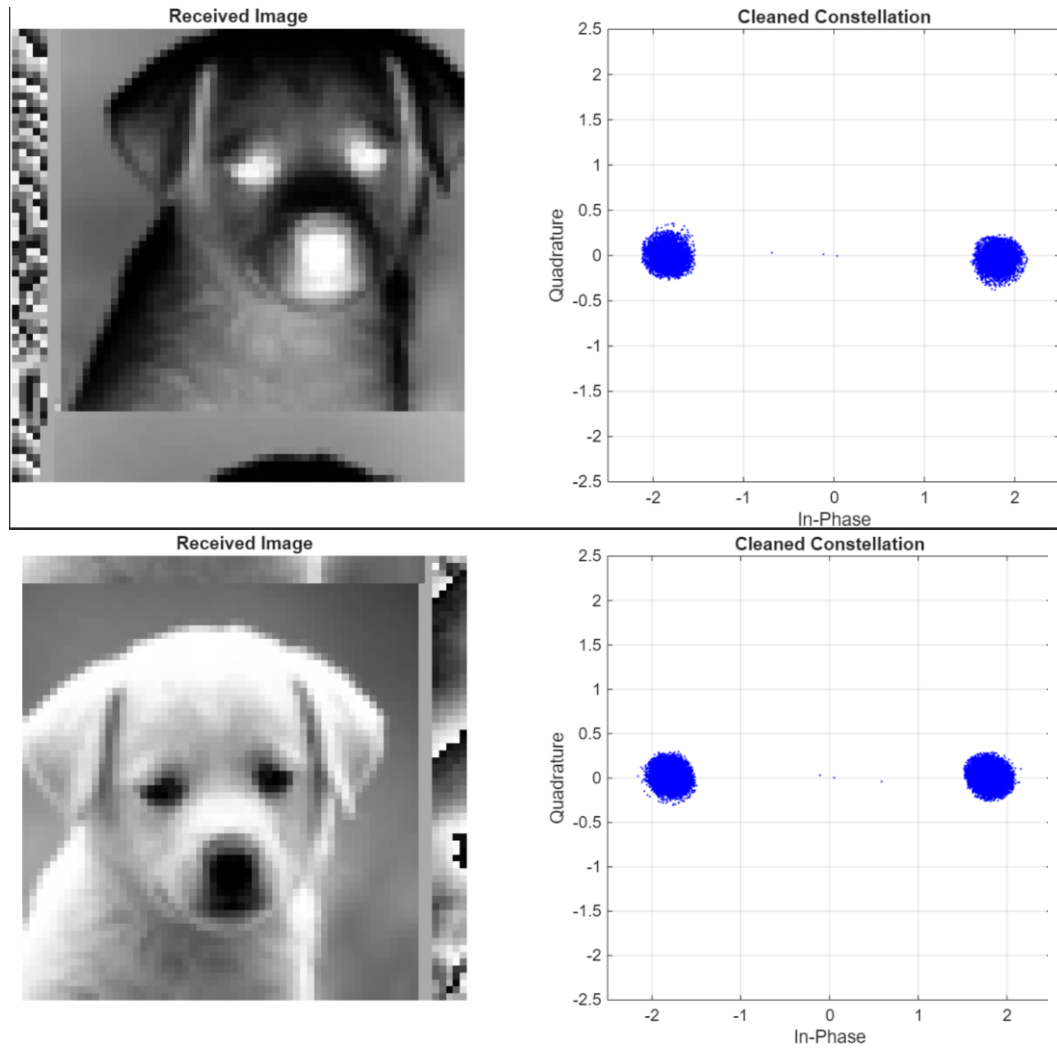


Fig 5: Air transmission (Antennas), different device

5.4 Simulation vs. Hardware Comparison

Metric	Simulation	ADALM-PLUTO
BER	0	Hardware dependent
Image Quality	Perfect reconstruction	Visible noise/artifacts
Channel Model	Pure AWGN (controlled)	Real RF + hardware impairments
Carrier Frequency	Baseband (none)	2.4 GHz
Sample Rate	100 kHz	2MHz
Modulation	BPSK + RRC	BPSK + RRC
Synchronization	Fixed offset	Preamble cross-correlation

The simulation represents the theoretical best-case performance upper bound, while the hardware results reflect the additional impairment budget of a real RF system. The gap between them quantifies the combined effect of hardware non-idealities, multipath, and the practical limitations of the simple carrier phase and IQ correction algorithms used.

5.5 Overall Observations

- **BER Sensitivity:** Image quality is highly sensitive to errors. Because errors are uniformly distributed, even a 1-2% BER produces noticeable pixel-level noise.
- **Synchronization is Critical:** Without preamble-based synchronization, hardware sampling fails entirely, resulting in randomized data regardless of signal strength.
- **System Validation:** The project successfully bridges theory and reality, proving that the simulated BPSK baseband algorithms can function over a physical, unguided RF link when properly augmented with carrier recovery and synchronization techniques.

6. CONCLUSION

This project successfully demonstrated the end-to-end transmission of an image using BPSK modulation. The purely simulated AWGN environment provided a mathematical baseline, proving that pulse shaping and basic detection work at sufficient SNRs. Transitioning to the ADALM-PLUTO SDR highlighted the realities of physical communication systems. While a wired SDR connection yielded near-perfect results, over-the-air wireless transmission introduced real-world RF impairments like phase rotation, fading, and interference. The addition of preamble-based synchronization and carrier phase correction in the SDR MATLAB code proved essential for successfully recovering the image in a physical environment.

7.1 MATLAB CODE (SIMULATION)

```
clc; clear; close all;
```

```
%% ===== SYSTEM PARAMETERS =====
```

```
Rb = 1e3;
```

```
Tb = 1/Rb;
```

```
Fs = 1e5;
```

```
Ns = round(Fs*Tb);    % Samples per bit
```

```
snr_values = 5:5:50;  % SNR range for analysis
```

```
%% ===== IMAGE PROCESSING =====
```

```
img = imread('C:\Users\nitin\OneDrive\Documents\MATLAB\B2DBy.jpg');
```

```
img = imresize(img,[64 64]);
```

```
if size(img,3)==3
```

```
    img = rgb2gray(img); % Convert to grayscale
```

end

```
img_vec = img(:);    % Serialize image
```

```
%% ===== BIT GENERATION =====
```

```
bits = de2bi(img_vec,8,'left-msb');
```

```
bits = bits.';
```

```
bits = bits(:);
```

```
bits = double(bits);
```

```
Nb = length(bits);
```

```
%% ===== BPSK MODULATION =====
```

```
symbols = 2*bits - 1;    % 0 → -1, 1 → +1
```

```
%% ===== RRC FILTER =====
```

```
rolloff = 0.35;
```

```
span = 6;
```

```
rrc = rcosdesign(rolloff, span, Ns);
```

```
%% ===== TRANSMITTER =====
```

```
tx_signal = upfirdn(symbols, rrc, Ns, 1); % Upsample + pulse shaping
```

```
tx_signal = tx_signal ./ max(abs(tx_signal));
```

```
%% ===== DISPLAY SETUP =====
```

```
figure;
```

```
subplot(3,4,1);
```

```
imshow(img);
```

```
title('Original');
```

```
plot_idx = 2;
```

```
%% ===== SNR LOOP =====
```

```
for snr = snr_values
```

```

%% Channel (AWGN)
rx_signal = awgn(tx_signal, snr, 'measured');

%% Receiver filtering
rx_filt = conv(rx_signal, rrc, 'same');

%% Remove filter delay
delay = span * Ns / 2;
rx_filt = rx_filt(delay+1:end);

%% Timing offset (non-ideal sampling)
offset = round(Ns/3);

%% Symbol sampling
sample_idx = offset:Ns:Nb*Ns;
sample_idx = sample_idx(sample_idx <= length(rx_filt));
rx_samples = rx_filt(sample_idx);

%% Detection
bits_rx = real(rx_samples) > 0;
bits_rx = bits_rx(:);

%% Length adjustment
if length(bits_rx) < Nb
    bits_rx(end+1:Nb) = 0;
elseif length(bits_rx) > Nb
    bits_rx = bits_rx(1:Nb);
end

%% BER calculation
BER = mean(bits ~= bits_rx);

```

```

%% Image reconstruction
bits_rx_matrix = reshape(bits_rx,8,[]).';
img_rx_vec = bi2de(bits_rx_matrix,'left-msb');
img_rx = reshape(uint8(img_rx_vec), size(img));

%% Display result
subplot(3,4,plot_idx);
imshow(img_rx);
title(['SNR=', num2str(snr), ', BER=', num2str(BER,3)]);
plot_idx = plot_idx + 1;

end

```

7.2 MATLAB CODE (ADALM-PLUTO same device)

```

clc; clear; close all;

%% ===== SETTINGS =====

Fc_rf = 2.4e9; Fs = 2e6; % increased sampling rate (IMPORTANT)

Rb = 10e3;

Ns = round(Fs/Rb);

txGain = -10;

rxGain = 25;

%% ===== IMAGE ===== img =
imread('C:\Users\nitin\OneDrive\Documents\MATLAB\B2DBY.jpg');

img = imresize(img,[64 64]);

if size(img,3)==3

img = rgb2gray(img);

end

```

```

img_vec = img(:);

bits = de2bi(img_vec,8,'left-msb');

bits = bits.';

bits = bits(:);

bits = double(bits);

Nb_payload = length(bits);

%% ===== PREAMBLE =====

preamble = repmat([1 0 1 0 1 0 1 0],1,50).';

preamble = double(preamble);

bits_tx = [preamble; bits];

%% ===== BPSK =====

symbols = 2*bits_tx - 1;

%% ===== RRC FILTER (KEY NOISE REDUCTION) =====

rolloff = 0.35;

span = 6;

rrc = rcosdesign(rolloff, span, Ns);

tx_base = upfirdn(symbols, rrc, Ns, 1);

tx_iq = tx_base + 1j*1e-12*randn(size(tx_base));

tx_iq = tx_iq ./ max(abs(tx_iq));

%% ===== PLUTO TX ===== txObj = sdrtx('Pluto', ...
'CenterFrequency', Fc_rf, ... 'BasebandSampleRate', Fs, ... 'Gain', txGain);

%% ===== PLUTO RX ===== rxObj = sdrx('Pluto', ...
'CenterFrequency', Fc_rf, ... 'BasebandSampleRate', Fs, ... 'SamplesPerFrame',
length(tx_iq)*2, ... 'GainSource','Manual', ... 'Gain', rxGain);

%% ===== TRANSMIT ===== transmitRepeat(txObj,
tx_iq); pause(1);

```

```

%% ===== RECEIVE ===== rx = rxObj();

release(txObj); release(rxObj);

rx = double(rx); rx = rx ./ max(abs(rx));

%% ===== STRONGER CARRIER RECOVERY =====
% BPSK phase correction (important improvement) ph = angle(mean(rx.^2))/2; rx = rx .* exp(-
1j*ph);

%% ===== MATCHED FILTER ===== rx_filt = conv(rx,
rrc, 'same'); rx_filt = rx_filt / sum(rrc.^2);

% simple smoothing (noise reduction) rx_filt = movmean(rx_filt,3);

%% ===== PREAMBLE SYNC ===== preamble_wave =
upfirdn(2*preamble-1, rrc, Ns, 1);

corr = conv(real(rx_filt), flipud(preamble_wave));

[~, peak] = max(abs(corr));

start = peak - length(preamble_wave) + 1; start = max(1,start);

first_symbol = start + length(preamble_wave);

%% ===== SAFE SAMPLING ===== sample_idx =
first_symbol : Ns : first_symbol + Ns*(Nb_payload-1); sample_idx = sample_idx(sample_idx
> 0 & sample_idx <= length(rx_filt));

Nb_valid = min(length(sample_idx), Nb_payload);

rx_samples      =      zeros(Nb_payload,1);      rx_samples(1:Nb_valid)      =
rx_filt(sample_idx(1:Nb_valid));

%% ===== DETECTION (ROBUST) ===== rx_real =
real(rx_samples);

threshold = 0.05 * mean(abs(rx_real)); % adaptive threshold bits_rx = rx_real > threshold;

bits_rx = double(bits_rx(:));

%% ===== LENGTH FIX ===== if length(bits_rx) <
Nb_payload bits_rx(end+1:Nb_payload) = 0; elseif length(bits_rx) > Nb_payload bits_rx =
bits_rx(1:Nb_payload); end

%% ===== BER ===== BER = mean(bits ~= bits_rx);
fprintf('BER = %.6f\n', BER);

```



```

%% ===== IMAGE RECONSTRUCTION =====
bits_rx_matrix = reshape(bits_rx,8,[]).'; img_rx_vec = uint8(bi2de(bits_rx_matrix,'left-msb'));
img_rx = reshape(img_rx_vec, size(img));

%% ===== DISPLAY ===== figure;

subplot(1,2,1); imshow(img); title('Original Image');

subplot(1,2,2); imshow(img_rx); title(['Pluto RX (BER = ', num2str(BER), ')']);

```

7.3 MATLAB CODE (ADALM-PLUTO different devices)

Transmitter :

```

%% ===== PC 1: TRANSMITTER =====

clc; clear; close all;

%% 1. PARAMETERS

Fs = 600e3;

Rb = 60e3;

Ns = Fs/Rb;

centerFreq = 915e6;

rolloff = 0.35;

span = 6;

%% 2. PREAMBLE & IMAGE PROCESSING

warmup_bits = repmat([1; 0], 500, 1);

warmup_sym = 2*warmup_bits - 1;

preamble_bits = [1 1 1 1 1 0 0 1 1 0 1 0 1];

preamble_sym = 2*preamble_bits - 1;

```

```

% Load and Resize Image

img = imread('C:\Users\nitin\OneDrive\Documents\MATLAB\B2DBy.jpg');

img = imresize(img, [64 64]);

if size(img,3) == 3

    img = rgb2gray(img);

end

img_vec = img(:);


% Convert image to bits

bits_tx = de2bi(img_vec, 8, 'left-msb');

bits_tx = double(bits_tx. ');

bits_tx = bits_tx(:); % These are the ground truth bits


% --- NEW: SAVE BITS FOR BER CALCULATION ---

% Copy this 'tx_bits.mat' file to the Receiver PC's current folder

save('tx_bits.mat', 'bits_tx');

fprintf('Ground truth bits saved to tx_bits.mat. Copy this to PC 2.\n');

% -----


data_sym = 2*bits_tx - 1;

tx_symbols = [warmup_sym; preamble_sym; data_sym];


%% 3. TRANSMITTER PIPELINE

txFilter = comm.RaisedCosineTransmitFilter(...

    'RolloffFactor', rolloff, 'FilterSpanInSymbols', span, ...

```

```

'OutputSamplesPerSymbol', Ns);

tx_signal = txFilter(tx_symbols);

tx_signal = tx_signal ./ max(abs(tx_signal));

tx_signal = complex(tx_signal);

%% 4. ADALM-PLUTO HARDWARE SETUP

disp('Initializing ADALM-PLUTO SDR Transmitter...');

tx = sdrtx('Pluto', 'CenterFrequency', centerFreq, ...

    'BasebandSampleRate', Fs, 'Gain', -10);

disp('Transmitting signal over the air...');

transmitRepeat(tx, tx_signal);

```

Reciever:

```

clc; clear; close all;
%% 1. PARAMETERS
Fs = 600e3;
Rb = 60e3;
Ns = Fs/Rb;
centerFreq = 915e6;
rolloff = 0.35;
span = 10;
img_dimensions = [64 64];
num_data_bits = img_dimensions(1) * img_dimensions(2) * 8;
% Define Preamble (Must match Tx \
% Define Robust Preamble (65-bit sequence for reliable Blind Sync)
barker_13 = [1 1 1 1 1 0 0 1 1 0 1 0 1];
preamble_bits = repmat(barker_13, 5, 1); % Repeats the sequence 5 times
preamble_sym = 2*preamble_bits - 1;
%% 2. HARDWARE CAPTURE
rx = sdrxx('Pluto', 'CenterFrequency', centerFreq, ...

```

```

'BasebandSampleRate', Fs, 'GainSource', 'Manual', 'Gain', 50, ...
'SamplesPerFrame', 2e6);
disp('Capturing signal...');
rx_signal_raw = rx(); release(rx);
rx_signal_raw = double(rx_signal_raw);
%% 3. SIGNAL PROCESSING CHAIN
% DC Block & Normalization
rx_signal = rx_signal_raw - mean(rx_signal_raw);
rx_signal = rx_signal / max(abs(rx_signal));
% Matched Filtering
rxFilter = comm.RaisedCosineReceiveFilter('RolloffFactor', rolloff, ...
'FilterSpanInSymbols', span, 'InputSamplesPerSymbol', Ns, 'DecimationFactor', 1);
rx_filt = rxFilter(rx_signal);
% Frequency & Phase Sync
coarseFreq = comm.CoarseFrequencyCompensator('Modulation', 'BPSK', 'SampleRate', Fs);
rx_coarse = coarseFreq(rx_filt);
symbolSync = comm.SymbolSynchronizer('SamplesPerSymbol', Ns,
'NormalizedLoopBandwidth', 0.01);
rx_sync = symbolSync(rx_coarse);
% Fine Phase Tracking (Costas Loop)
carrierSync = comm.CarrierSynchronizer('Modulation', 'BPSK', 'NormalizedLoopBandwidth',
0.005);
rx_final = carrierSync(rx_sync);
%% 4. BLIND FRAME SYNC & DECODING (Energy Detection + Correlation)
disp('Performing blind synchronization...');
% Step 1: Energy Detection (Find the actual signal burst in the noise)
window_size = 500;
signal_power = movmean(abs(rx_final).^2, window_size);
threshold = 0.4 * max(signal_power); % Trigger at 40% of max power
burst_start = find(signal_power > threshold, 1, 'first');
if isempty(burst_start) || (burst_start + num_data_bits > length(rx_final))
disp('ERROR: Incomplete or missing signal. Make sure PC 1 is transmitting *DURING*
capture.');
```

return; % Safely stop the script

end

```

% Step 2: Restrict correlation search to the start of the burst
search_range = burst_start : min(length(rx_final), burst_start + 2000);
search_window = rx_final(search_range);
[corr, lags] = xcorr(search_window, preamble_sym);
[~, max_idx] = max(abs(corr));
% Step 3: Calculate the exact data payload start index
start_idx = search_range(1) + lags(max_idx) + length(preamble_sym);
if start_idx > 0 && (start_idx + num_data_bits - 1) <= length(rx_final)
disp('Preamble successfully locked! Extracting image...');
```

% Extract the raw complex symbols

```

raw_syms = rx_final(start_idx : start_idx + num_data_bits - 1);
% Step 4: Phase Ambiguity Correction
if real(corr(max_idx)) < 0
disp('180-degree phase inversion detected. Correcting...');
```

raw_syms = -raw_syms;

```

end
% Hard Decision
bits_rx = real(raw_syms) > 0;
else
disp('ERROR: Sync Failed. Burst detected but preamble alignment is out of bounds.');
```

return; % Safely stop the script

```

end
%%% 5. IMAGE RECONSTRUCTION & DISPLAY
% Ensure bit array is perfectly sized for reshaping
bits_rx = bits_rx(1:num_data_bits);
% Reconstruct the image
img_rx = reshape(uint8(bi2de(reshape(bits_rx, 8, []).', 'left-msb')), img_dimensions);
% Display Results
figure;
% --- LEFT PANEL: IMAGE ---
subplot(1,2,1);
imshow(img_rx);
title('Received Image');
```

% --- RIGHT PANEL: CONSTELLATION ---

```

subplot(1,2,2);
plot(real(raw_syms), imag(raw_syms), 'b.', 'MarkerSize', 2);
title('Cleaned Constellation');
xlabel('In-Phase');
ylabel('Quadrature');
grid on;
axis([-2.5 2.5 -2.5 2.5]); % Lock axes so the scale matches
axis square;
```